LangChain 체인 학습 노트

날짜: 2026-05-22 복습 예정: D+1, D+3, D+7

한 줄 요약

LLM에게 질문할 때 "질문 템플릿 → LLM 호출 → 결과 파싱"을 파이프라인처럼 연결하는 도구.

---

1. 비유

공문서 작성에 비유: 1. 양식지**(Prompt Template)**: 빈칸만 채우면 되는 공문서 양식 2. 담당자**(LLM)**: 양식 받아서 검토하고 결재 3. 결재 결과 파싱**(OutputParser)**: 결재문서에서 필요한 정보만 뽑아냄

```
[양식 작성] → [담당자 검토] → [결과 추출]
  Prompt      LLM 호출       OutputParser
```

---

2. 개념

> LangChain은 LLM 호출을 파이프라인(체인)으로 구성하는 프레임워크. | 연산자로 컴포넌트를 연결한다 (UNIX 파이프와 같은 개념).

주요 컴포넌트:

컴포넌트	역할	Dev 2 예시
`ChatPromptTemplate`	질문 템플릿 정의	"이슈: {message}, severity: {current_severity}..."
`ChatOpenAI`	OpenAI API 호출	gpt-4o-mini 호출
`StrOutputParser`	LLM 응답을 문자열로 파싱	JSON 문자열 추출

---

3. 왜 쓰나

- 없으면: `openai.chat.completions.create(...)` 매번 직접 호출 → 반복 코드 많고 관리 어려움
- 있으면: 체인으로 한 번 정의하면 재사용 가능, LangSmith 자동 추적

---

4. 코드 (Dev 2)

```python
# nodes/llm_judge.py
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

# 1) 템플릿 정의 (빈칸 있는 양식)
prompt = ChatPromptTemplate.from_messages([
    ("system", "당신은 시니어 보안 엔지니어입니다. JSON으로만 답하세요."),
    ("human", "이슈: {message}\n현재 severity: {current_severity}\n개발자 수준:
{developer_level}")
])

# 2) LLM 설정
llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)

# 3) 체인 조립 (파이프 | 연산자로 연결)
chain = prompt | llm | StrOutputParser()

# 4) 실행
response = await chain.ainvoke({
    "message": "SQL 인젝션 가능성",
    "current_severity": "Major",
    "developer_level": "junior",
})
# response → '{"severity": "Critical", "suggestion": "파라미터화된 쿼리 사용하세
요."}'
```

핵심: | 연산자가 파이프처럼 왼쪽 출력 → 오른쪽 입력으로 전달

Minor 스킵 전략 (비용 최소화)

```python
# Minor 이슈는 LLM 호출 안 함 → API 비용 절약
if issue["severity"] == "Minor":
    enriched.append(issue)  # 그대로 통과
    continue

# Major/Critical만 LLM 재판단
response = await chain.ainvoke(...)
```

복습 퀴즈

1. Q: LangChain 체인에서 | 연산자의 역할은? A: (왼쪽 컴포넌트의 출력을 오른쪽 컴포넌트의 입력으로 전달)

2. Q: Dev 2에서 LLM을 Minor 이슈에 호출하지 않는 이유는? A: (API 비용 최소화. Minor는 CLI 결과 그대로 사용)
3. Q: ChatPromptTemplate에서 `{message}` 같은 중괄호의 역할은? A: (나중에 실제 값으로 채워질 빈칸 변수)